VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software |

Unit Coverage: 1 – 5 Max

Marks: 100

Chosen Software: GIT

Student Name	Registration Number
Yathartha Rastogi	**24BCE10410**
Slot	**Date of Submission**
D11	**31-03-2026**

1. Introduction

Most digital systems which operate the current world were developed through open access rather than restricted laboratory environments. The open-source philosophy changes the software development process by transforming it from a profit-oriented business model into a worldwide collaborative work process. The system guarantees that all developers worldwide can freely share and study and modify and enhance the essential tools which people depend on.

I selected Git as the subject for my open-source audit project. The distributed version control system Git enables developers to track code changes while they work together in a secure environment. Today, it stands as the undisputed backbone of modern software engineering and is deeply embedded in almost every collaborative development environment globally.

Git was originally forged out of pure necessity to solve critical bottlenecks during the development of the Linux kernel. Its development progressed rapidly beyond its initial design because of its exceptional speed and data protection capabilities and its adaptable system design. The open-source license protects the software which enables its worldwide developer community to work on its improvement and security development.

The report will examine Git through different aspects to assess its complete effect on software development. The study investigates the system's beginning and philosophical foundations while examining its license rights and operational effects on Linux platforms and its essential function in open-source software development and its comparison to proprietary software solutions.

I created five shell scripts to show practical Linux administration skills which I documented in my written analysis. The scripts provide users with a practical demonstration of open-source command-line tools which they can use to see how these tools function in actual situations.

2. Choosing Your Software

Your choice	Write your chosen software here before submission: Git

Part A — Origin and Philosophy (Units 1 & 2)

A1. Origin Story

The software development process encountered a fundamental restriction which Git later removed. The increasing complexity of projects required multiple developers to work together in shared codebases. The first version control systems existed to support collaborative work but their main function was to operate from a central location. The system architecture required developers to access a single central server. This limited their development capabilities. The system operated at slow speeds while developers faced extreme difficulties with code branching and merging processes. This required them to maintain a constant network connection for both project history access and work submission.

The Linux kernel project stands as the largest collaborative software project worldwide and needed to develop an improved system that could handle its extensive operational requirements. The Linux development community used the proprietary BitKeeper software as their primary tool during its initial development phase. The Linux community received free access to BitKeeper yet the software did not qualify as true open source. The arrangement between the two parties collapsed when Linux developers and BitKeeper's parent company entered into licensing conflicts which caused the company to withdraw its free access rights to the community.

The crisis revealed an enormous contradiction because the primary open-source operating system of the world relied completely on a proprietary software tool which it depended on heavily for development. Linus Torvalds, who created Linux, recognized this dependency as a threat and thus decided to develop an entirely new version control system which he wanted to build from scratch. He aimed to create a system that would exceed BitKeeper's capabilities while addressing the critical problems present in existing systems. The system he designed needed to operate at maximum speed while using a distributed framework and maintaining strong safeguards against data loss and supporting large complex projects such as the Linux kernel.

The result was Git, developed and released in 2005. Git revolutionized the industry by introducing a highly efficient Distributed Version Control System (DVCS). Git allowed every developer to have a complete local copy of the repository, unlike centralized systems. Developers had the ability to work without an internet connection while performing rapid operations and handling complex branching and merging tasks that required no access to a central server.

Torvalds made an important decision by releasing Git through the open-source GPL v2 license. The licensing decision emerged as a direct result of the BitKeeper conflicts. His decision to make Git available as open-source software prevented any single company from obtaining control of the tool and stopping users from accessing it while they could develop it without restriction. The software allowed developers throughout the world to use it freely while they could modify it and distribute it permanently.

Git has expanded its capabilities from its initial development as a specialized tool for Linux kernel development. Git has become the worldwide standard for managing version control. Git provides version control solutions to the entire technology industry through its support of large collaborative platforms such as GitHub and GitLab that are used by developers, startups, and enterprise companies. The developers of Git created a permanent solution to a licensing dispute which fundamentally changed how software is developed and managed worldwide.

A2. License: what it actually says

Git is officially licensed under the GNU General Public License version 2 (GPL v2). The particular license functions as the legal mechanism that protects Git from becoming a commercial product instead of remaining a public resource.

The Four Freedoms

The free software movement's core principle exists in the GPL v2 license through its four established rights. The contents of the document serve as essential user rights which transfer decision-making power from developers to their users. The following elements are present in the document:

- **The freedom to run** the software however you want, for any purpose.
- **The freedom to study** how the software works by having full access to its source code.
- **The freedom to modify** the software to suit your specific needs.
- **The freedom to distribute** copies of the software (and your modifications) to help others.

The system guarantees that software functions as an accessible tool which users can both comprehend and enhance. The system guarantees that users will have multiple options to address bug fixes and new feature requests instead of relying on one particular company.

A common question regarding open-source software is whether a company can take it, modify it, and sell it. With Git, the answer is yes, but with major conditions.

Because GPL v2 is a "copyleft" license, it requires complete distribution of all derived works. A company can absolutely build and sell a product using modified Git code. However, they are legally required to release the source code of their modifications under the exact same GPL v2 license. The copyleft mechanism functions perfectly because it safeguards all future versions of software which developers create. The system stops companies from transforming open-source community software into exclusive, profit-driven proprietary products.

When looking at the broader open-source ecosystem, licenses usually fall into two categories: strict and permissive. The difference between the GPL and the MIT license perfectly illustrates this.

- **GPL (Strict/Copyleft):** Forces anyone who distributes modified versions to share their changes. It fiercely protects user freedom downstream.
- **MIT (Permissive):** Allows developers to take the code, modify it privately, and even bake it into closed, proprietary software without ever sharing their improvements.

I would select the GPL as my preferred license for my project. The GPL license protects essential project openness while MIT license permits companies to use the project without restrictions. The system requires all users who gain benefits from the software to share their enhancements with the community.

The English language makes open source slightly confusing because the word "free" has two distinct meanings. The first meaning defines "free" as "free as in free beer" which means users can download the software at no cost. The second meaning defines "free" as "free as in freedom" which grants users full control over their software rights.

Git offers free access to its software because both its first and second meanings of free software licensing apply. The true power of Git exists because users can study and modify and distribute the system without any financial constraints. Organizations can modify Git software through open-source access when developers discover that its standard functions do not match their specific enterprise needs. The developers control their work because they no longer need to wait for vendor approval to proceed.

The GPL v2 license of Git functions as its primary legal framework which ensures the software will always operate as a community-driven resource that corporate interests cannot control.

A3. The ethics of open source

Open-source software operates as a licensing system which establishes rules for sharing knowledge and developing software tools. The system requires us to examine its legal aspects while we investigate essential issues regarding equity and work and technological progress.

1. Should All Software Be Open Source?

The question of whether all software should become open source needs to be evaluated through two different perspectives. The complete opening of all software code would create an equal technological playing field for all users. The public accessibility of code enables faster community-driven security updates because security flaws become visible to all users. The world gains faster innovation because developers share their work to help others who want to enhance their knowledge. Open-source software enables students and startups to develop innovative concepts because it eliminates the need to pay for costly proprietary software.

A world that operates under complete open-source principles proves to be unworkable. Businesses require dependable income sources to continue operations yet they cannot distribute all their software products at no cost while keeping their business model operational. The security risks establish valid concerns because malicious actors obtain complete access to system vulnerabilities through open code which good actors can use for auditing purposes. The absence of financial incentives will prevent development of complex projects which need specific expertise because contributors need paid work to support themselves.

2. Is It Ethical for Companies to Profit from Open Source?

This brings up a heavily debated question: Is it ethical for a company to build a profitable product entirely on top of free community labour? I believe the answer to the question is yes although the result depends on the company's actions. Open-source licenses exist to permit commercial usage while businesses create additional worth through their enterprise support and user-friendly interfaces and dependable cloud infrastructure.

Massive corporations create an ethical problem when they take substantial community project value without providing any benefits to the project. The system creates a major imbalance because unpaid volunteers support essential infrastructure while a company generates profits from their unpaid efforts. The use of open source by corporations becomes ethical when a company provides active support through monetary donations or code contributions or employment of main developers but turns unethical when they function as value-stealing entities.

3. "Standing on the Shoulders of Giants"

The term "standing on the shoulders of giants" in software development means that current developers can use existing work instead of building new software from nothing. Every new application is built upon the foundational work of thousands of

programmers who came before them. Git serves as an ideal case study because Linus Torvalds used existing version control methods to create a far better system.

The open-source model through which I ask my question about original innovation leads me to conclude that it helps drive innovation forward. Developers who need to create new solutions to complex problems can dedicate their complete creative resources when they stop wasting time to build basic tools like databases and web servers. Open-source shows that cumulative innovation does not decrease originality because it provides innovators with extensive advantages at the start.

4. Conclusion

The open-source movement serves as the foundation which built the current internet system, but it has limitations as a complete solution. The technology sector depends on open-source software through the benefits provided by the open-source community. The current situation requires us to establish a new method which protects code ownership rights while providing developers with financial support for their work.

Part B — Linux Footprint (Unit 2)

1. Installation Method

Linux systems provide complete access to Git which users can install through standard package managers that include apt and dnf and yum. On Debian-based distributions, for instance, it can be installed in seconds using a simple command like sudo apt install git. The installation process through package managers provides a reliable method because it delivers secure verified software which has undergone extensive testing. Users who seek full control over their system should consider building Git from its original source code. The open-source tools offer users two distinct methods which enable them to choose between flexible and accessible options.

2. Key Directories

The primary Git software component installs to /usr/bin/git which establishes its presence as part of the system's default binary installation. A developer's global, user-specific settings are neatly stored in a configuration file located at ~/.gitconfig. The most important function of Git creates a hidden .git/ directory which it sets up inside every new project that gets started. This directory functions as the central control unit which stores all commit records and branch details and log files and system information. Git maintains all essential information on local storage systems which enables it to operate as a fully distributed version control system without needing any remote server access. The local system design enables developers to perform their work without internet access while achieving much faster performance than traditional methods.

3. User and Group

Git operates under the active user's access rights because its developers designed the program to function without requiring root or administrative access rights. The system achieves better security because the program only accesses files that belong to the user thereby blocking all other system access. Git maintains

fundamental security through its design which does not require users to obtain higher access rights.

4. **Service Management**

Git operates as a command-line tool that users activate whenever they require it. The system uses Git through its command-line interface which starts and stops with each operation. It requires this design because it needs to create a system that operates with minimal resource usage while maintaining high performance.

5. **Update Model**

The update model for Git depends completely on the capabilities of its open-source ecosystem. The standard Linux package managers provide seamless delivery of routine updates which the system requires. The updates which the system receives on a regular basis include important bug fixes and security updates and enhancements to system performance. The system demonstrates how open-source software maintenance requires multiple teams to work together in an interconnected environment.

Conclusion

Git provides complete integration with the Linux operating system which functions as its main environment. The system exhibits core open-source values through its complete transparent operating system design which treats all components as separate units and gives users total system command.

Part C — The FOSS Ecosystem (Units 3 & 4)

When it comes to open-source ecosystems, Git is the most valuable player.

The platforms GitHub, GitLab and Bitbucket use this feature to enable developers worldwide to work together on their projects. The operation of Git depends on basic system tools which it requires but it also creates opportunities for various applications and services to function.

There is a large community of developers who contribute to Git. The developers work on three main tasks which include fixing bugs and adding features and improving performance. Web-based code hosting platforms serve as the primary connection point between Git and its online environment.

Part D — Open Source vs Proprietary (Critical Analysis)

Dimension	Open Source	Proprietary Alternative
Cost	Git is completely free to download, use and modify.	Proprietary tools often require paid licenses, specifically for enterprise use.
Security (who can audit the code?)	Thanks to the public source code, vulnerabilities can be eliminated or pointed out by anyone.	The source code is closed, so only the company personnel can check and fix any issues.
Support and reliability	Backed by a large global community with great lengths documentation, large forums and regular updates.	Provides customer support, but depends on a subscription-based model and company response time.
Freedom to modify	Users can freely modify Git and create their own versions as per their needs.	Users cannot legally modify the software and must follow company-imposed restrictions.
Community vs corporate control	Developed and maintained by a global open-source community where contributions are shared and encouraged.	Controlled by a company, where decisions are driven by business goals and code is closed-source.
Your overall verdict	Git is free of cost, flexible and widely used, making it the ideal solution for developers and learners.	Proprietary tools may offer structured support but lack flexibility and freedom.

Conclusion

From my perspective, Git is a better choice for most developers because it provides free access and flexible usage together with extensive community support. The system provides complete code control to users while it promotes teamwork among developers.

The proprietary tools offer specialized assistance to users but their high expenses and limited usage rights together with restricted customization options make them less accessible.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

```
#!/bin/bash
STUDENT_NAME="Yathartha Rastogi"
SOFTWARE_CHOICE="Git"
KERNEL=$(uname -r)
USER_NAME=$(whoami)

if command -v uptime >/dev/null 2>&1; then
    UPTIME=$(uptime -p)
else
    UPTIME="Not available"
fi

if [ -f /etc/os-release ]; then
    DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')
else
    DISTRO="Windows (Git Bash)"
fi

DATE=$(date "+%d-%m-%Y %H:%M:%S")

echo "================================"
echo " Open Source Audit Report"
echo "================================"
echo "Name       : $STUDENT_NAME"
echo "Software   : $SOFTWARE_CHOICE"
echo "Kernel     : $KERNEL"
echo "User       : $USER_NAME"
echo "Uptime     : $UPTIME"
echo "OS         : $DISTRO"
echo "Date       : $DATE"
echo "License    : Linux follows GPL"
echo "================================"
```

```
yarag@MSI MINGW64 /d/SEMs/SEM5/OSS/Submission stuff/Scripts
$ ./script1.sh
================================
 Open Source Audit Report
================================
Name       : Yathartha Rastogi
Software   : Git
Kernel     : 3.6.5-22c95533.x86_64
User       : yarag
Uptime     : Not available
OS         : Windows (Git Bash)
Date       : 31-03-2026 20:30:04
License    : Linux follows GPL
================================
```

Script 2 — FOSS Package Inspector (Unit 2)

```
#!/bin/bash
PACKAGE="git"
if rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    rpm -qi $PACKAGE | grep -E 'Version | License | Summary'
else
    echo "$PACKAGE is NOT installed."
fi
case $PACKAGE in
    httpd) echo "Apache: the web server that built the open internet" ;;
    mysql) echo "MySQL: open source at the heart of millions of apps" ;;
    git) echo "Git: The tool Linus built when proprietary failed him" ;;
    vlc) echo "VLC: Plays anything - built by students in Paris" ;;
esac
```

```
yarag@MSI MINGW64 /d/SEMs/SEM5/OSS/Scripts
$ ./script2.sh
Checking for git...
------------------------
git is installed.
Version: git version 2.52.0.windows.1
Git enables distributed version control and collaboration.
```

Script 3 — Disk and Permission Auditor (Unit 2)

```
#!/bin/bash
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
echo "Directory Audit Report"
echo "----------------------"
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done
if [ -d "$HOME/.git" ]; then
    PERMS=$(ls -ld "$HOME/.git" | awk '{print $1, $3, $4}')
    echo "$HOME/.git => Permissions: $PERMS"
fi
```

```
yarag@MSI MINGW64 /d/SEMs/SEM5/OSS/Submission stuff/Scripts
$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x yarag 197609 | Size: 1.6M
/var/log does not exist on this system
/home does not exist on this system
/usr/bin => Permissions: drwxr-xr-x yarag 197609 | Size: 91M
/tmp => Permissions: drwxr-xr-x yarag 197609 | Size: 360M
```

Script 4 — Log File Analyzer (Units 2 & 5)

```
#!/bin/bash
LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
tail -n 5 "$LOGFILE" | grep -i "$KEYWORD"
```

```
yarag@MSI MINGW64 /d/SEMs/SEM5/OSS/Submission stuff/Scripts
$ ./script4.sh script1.sh echo
Keyword 'echo' found 11 times in script1.sh
echo "Uptime      : $UPTIME"
echo "OS          : $DISTRO"
echo "Date        : $DATE"
echo "License     : Linux follows GPL"
echo "==============================="
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

```
#!/bin/bash
echo "Answer three questions to generate your manifesto."
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"
echo "Open source for me means $FREEDOM." > $OUTPUT
echo "Using tools like $TOOL helps developers grow." >> $OUTPUT
echo "I want to build $BUILD and share it openly." >> $OUTPUT
echo "Date: $DATE" >> $OUTPUT
echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

```
yarag@MSI MINGW64 /d/SEMs/SEM5/OSS/Submission stuff/Scripts
$ ./script5.sh
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? liberty
3. Name one thing you would build and share freely: apps
Manifesto saved to manifesto_yarag.txt
Open source for me means liberty.
Using tools like Git helps developers grow.
I want to build apps and share it openly.
Date: 31 March 2026
```